Πίνακες Πολλαπλών Διαστάσεων

(Multidimensional arrays)

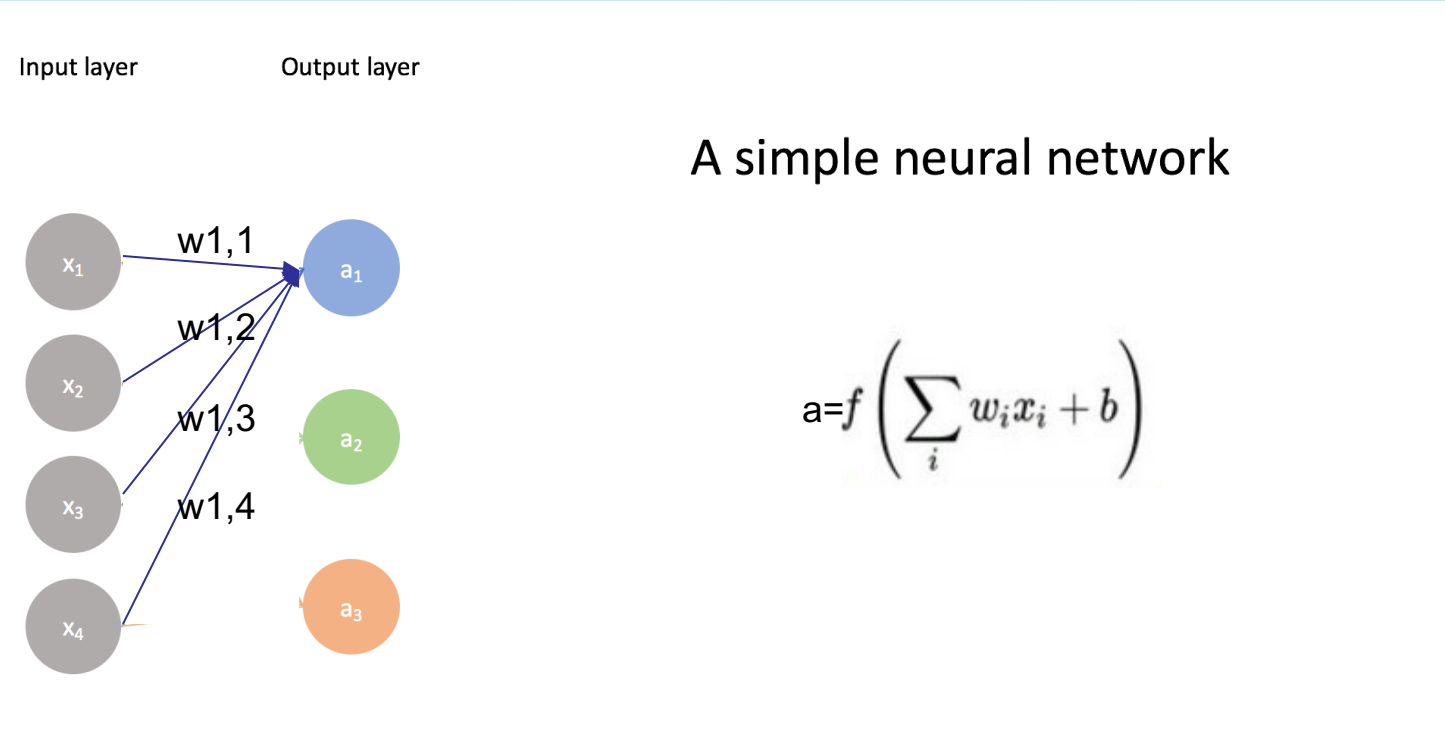
Έχουμε ήδη συζητήσει πίνακες μιας διάστασης

Είδαμε ότι οι πίνακες αντιπροσωπεύονται από τις διευθύνσεις τους -

Λαμβάνονται ως ορίσματα από κλήσεις συναρτήσεων ή/και επιστρέφονται από αυτές ως διευθύνσεις

Είδαμε επίσης παραδείγματα παράλληλων πινάκων

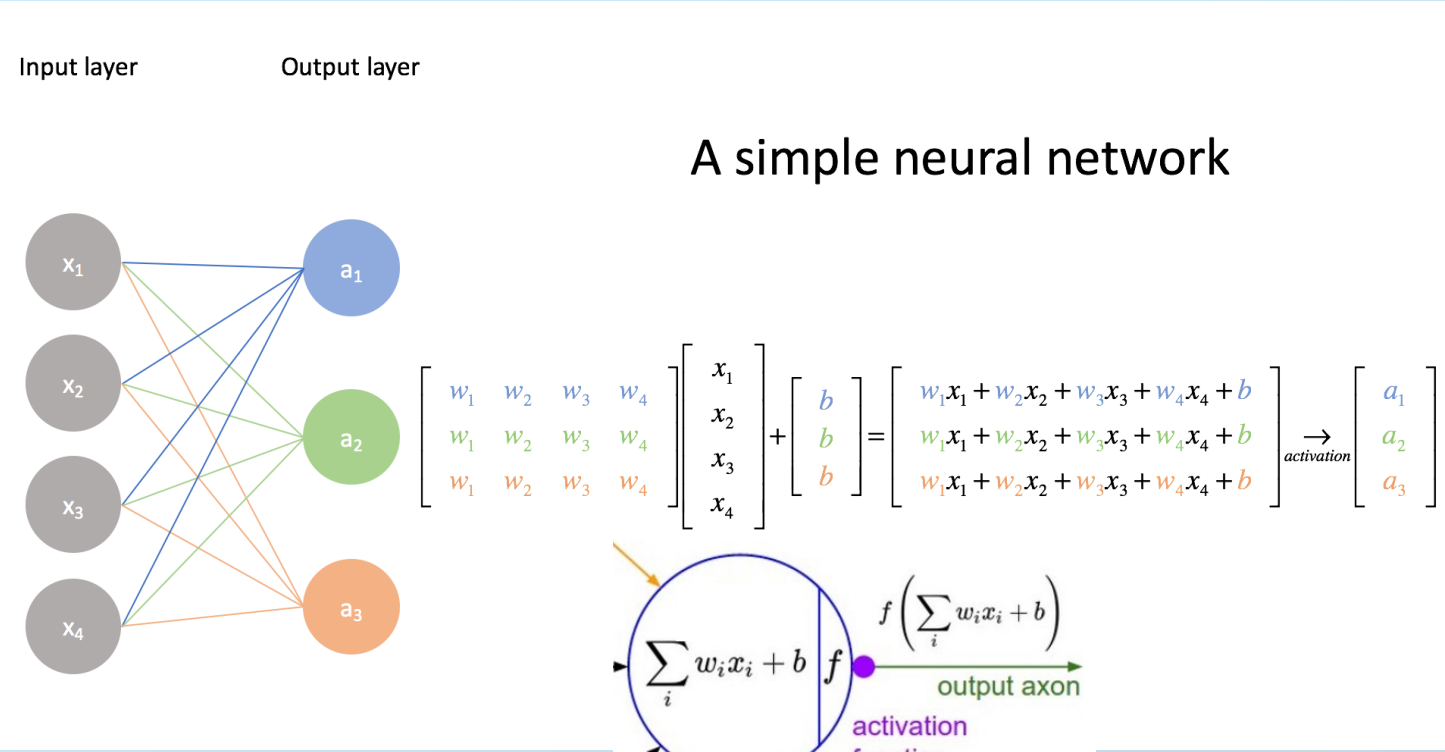
AI βάσει Μηχανικής Μάθησης και Νευρωνικών Δικτύων

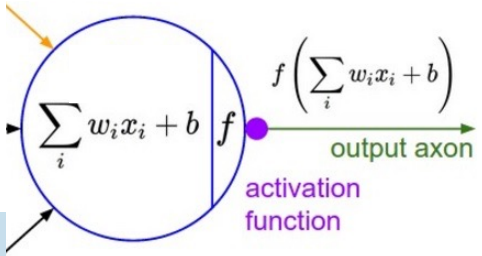


AI βάσει Μηχανικής Μάθησης και Νευρωνικών Δικτύων

Πολυδιάστατοι πίνακες για αναπαράσταση βαρών

Μονοδιάστατοι πίνακες για αναπαράσταση διανυσμάτων

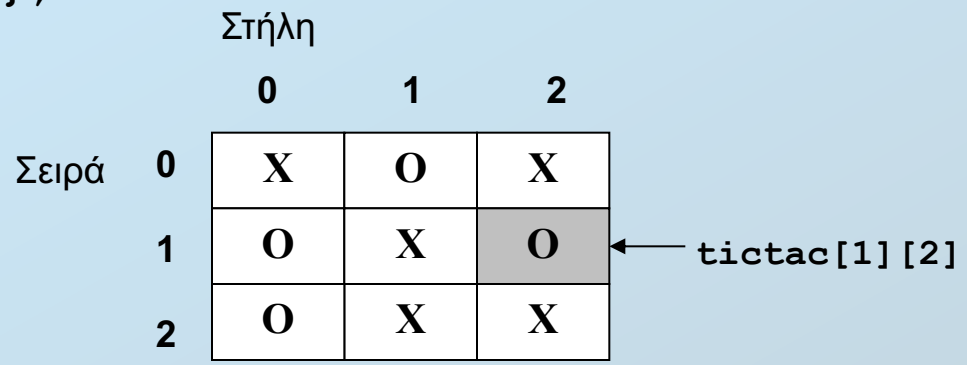




Πολυδιάστατοι Πίνακες – Multidimensional arrays

Ένας πίνακας είναι πολυδιάστατος αν έχει δύο ή περισσότερες διαστάσεις

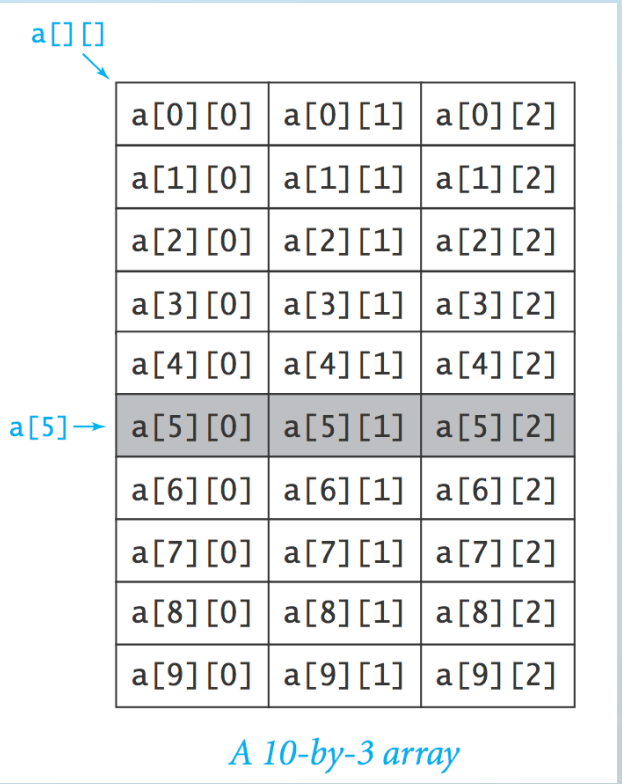
```
char[][] tictac = { {'X','O','X'},     // ρητή στατική αρχικοποίηση 3x3
                    {'O','X','O'},
                    {'O','X','X'}
                  };
```



Δυσδιάστατοι Πίνακες σε Java

Πρόσβαση στον πίνακα: Χρησιμοποιούμε την έκφραση **`a[i][j]`** για πρόσβαση στο στοιχείο που βρίσκεται στη σειρά **`i`** και στη στήλη **`j`**. Η αρίθμηση των σειρών και στηλών αρχίζει από το **`0`**.

```
int M = 10;
int N = 3;
double[][] a = new double[M][N];
for (int i = 0; i < M; i++) {
   for (int j = 0; j < N; j++) {
      a[i][j] = 0.0;
   }
}
```

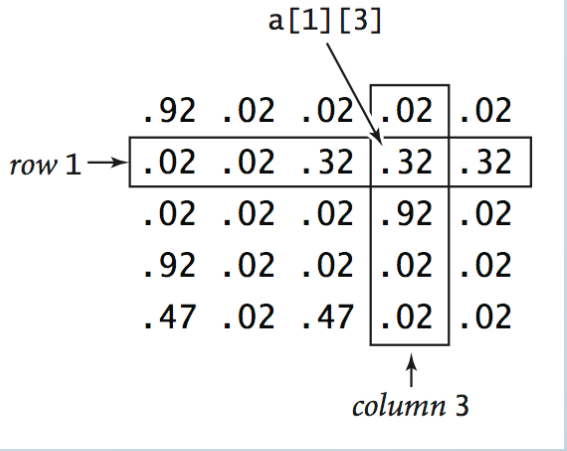


A 10-by-3 array

Αρχικοποίηση Δισδιάστατου Πίνακα

Καταγράφοντας ρητά τις τιμές, σειρά με σειρά

```
double[][] p =
{
    { .92, .02, .02, .02, .02 },
    { .02, .02, .32, .32, .32 },
    { .02, .02, .02, .92, .02 },
    { .92, .02, .02, .02, .02 },
    { .47, .02, .47, .02, .02 },
};
```



```
double[][][] table = new double[7][5][6];
```

1. Ο πίνακας table έχει τρεις διαστάσεις και σύνολο 210 στοιχεία. Οι δείκτες της πρώτης διάστασης είναι 0..6, της δεύτερης 0..4 και της τρίτης 0..5

2. table[2][3][4] σημαίνει το στοιχείο του πίνακα table που έχει δείκτη 2 στη πρώτη διάσταση (δηλαδή το τρίτο σε σειρά), έχει δείκτη 3 στη δεύτερη διάσταση και δείκτη 4 στη τρίτη διάσταση

1. Όλα τα στοιχεία του πίνακα table αρχικοποιούνται αυτόματα στην τιμή 0.0

Τρισδιάστατοι Πίνακες σε Java

```
int N = 5;
String[][][] a = new String[N][N][N];
//default value for string table element is null
// null is a literal
```

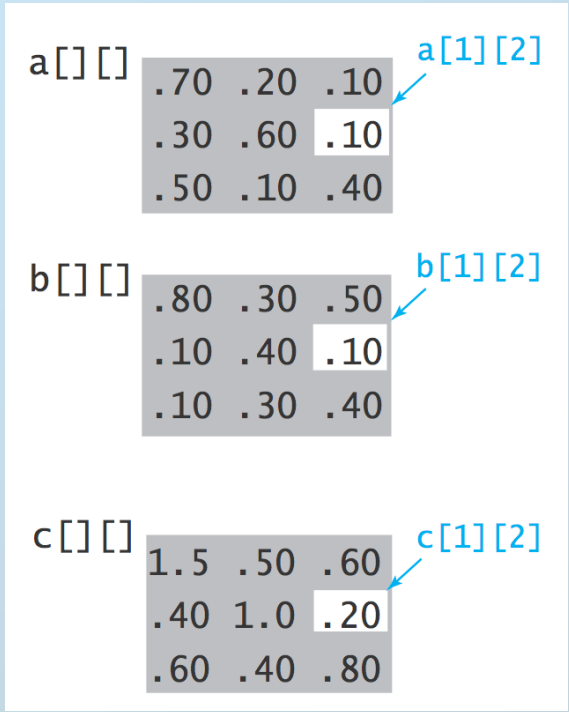
```
int N = 5;
String[][][] a = new String[N][N][N];

for (int i = 0; i < N; i++) {
   for (int j = 0; j < N; j++) {
        for (int k = 0; k < N; k++) {
            a[i][j][k]= ""; //empty string is not same as a[i][j][k]=null;
   }
}
```

Παράδειγμα: Πρόσθεση Τετραγωνικών Πινάκων, N × N

όλες οι τιμές αρχικοποιούνται στο 0

```
double[][] c = new double[N][N];
for (int i = 0; i < N; i++)
   for (int j = 0; j < N; j++)
      c[i][j] = a[i][j] + b[i][j];
```



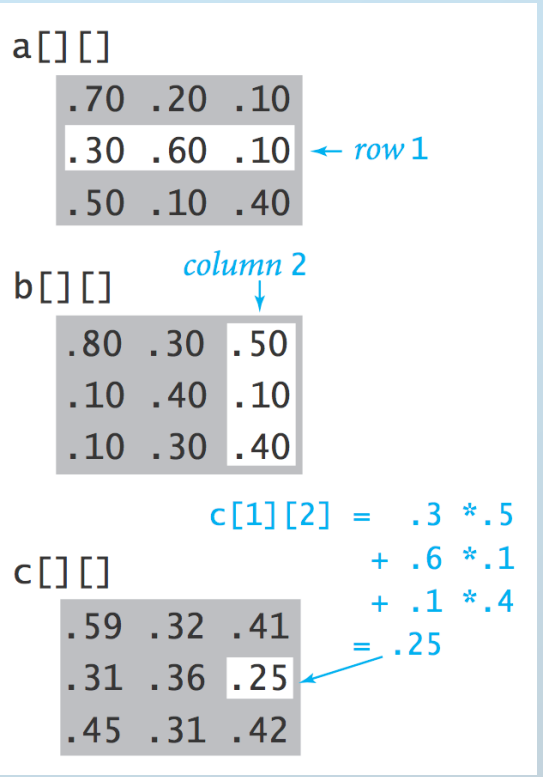
Παράδειγμα: Πολλαπλασιασμός Τετραγωνικών Πινάκων, N × N

όλες οι τιμές αρχικοποιούνται στο 0

```
double[][] c = new double[N][N];
for (int i = 0; i < N; i++)
   for (int j = 0; j < N; j++)
      for (int k = 0; k < N; k++)
         c[i][j] += a[i][k] * b[k][j];
```

Πόσες φορές εκτελείται ο τελεστής * για το γινόμενο δύο πινάκων;

A. N B. N^2 C. N^3 D. N^4



Παράδειγμα: Είναι δεδομένος πίνακας (σε σχέση με το παιγνίδι tic-tac-toe) τελείως γεμάτος;

X	O	E
E	O	E
X	E	E

Όχι

X	O	O
O	O	X
X	X	O

Ναι

```
public class Tables{

  public static boolean filled (char[][] ttt){
    for (char[] cs : ttt){ // for each row
      for (char c : cs){   // for each element of the row
         if (c == 'E') return false;
      }
    }
    return true;
  }

  public static void main(String[] args){
    char[][] ttt1 = {{'X', 'O', 'E'}, {'E','O','E'},
                     {'X','E','E'}};
    char[][] ttt2 = {{'X', 'O', 'O'}, {'O','O','X'},
                     {'X','X','O'}};
    System.out.println(filled(ttt1));
    System.out.println(filled(ttt2));
  }
}
```

```
$ java Tables
false
true
```

Παράδειγμα: Είναι δεδομένος πίνακας (σε σχέση με το παιγνίδι tic-tac-toe) τελείως κενός;

```
public static boolean empty (char[][] ttt){
  for (char[] cs : ttt){ // for each row of ttt
    for (char c : cs){   // for each element of the row
       if (c != 'E') return false;
    }
  }
  return true;
}
```

```
char [][] table = new char [3][3];
```

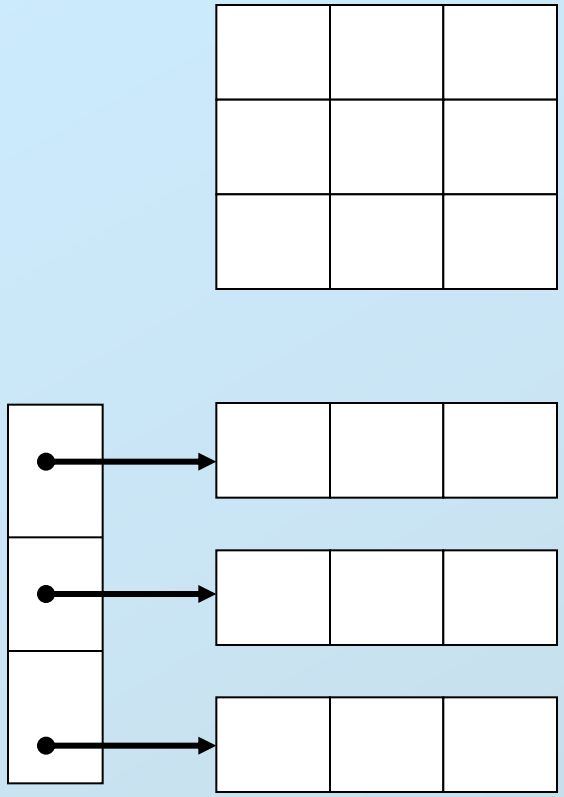


table ισοδυναμεί με τη διεύθυνση του πρώτου στοιχείου (**table[0][0]**)

table[0] ισοδυναμεί με τη διεύθυνση της πρώτης σειράς του πίνακα, δηλαδή και πάλι με τη διεύθυνση του στοιχείου **table[0][0]**

table[1] ισοδυναμεί με τη διεύθυνση της δεύτερης σειράς του πίνακα, δηλαδή με τη διεύθυνση του στοιχείου **table[1][0]**

table[2] ισοδυναμεί με τη διεύθυνση της τρίτης σειράς του πίνακα, δηλαδή με τη διεύθυνση του στοιχείου **table[2][0]**

Σειρές και Στήλες

Επισημάνσεις σε σχέση με τύπους

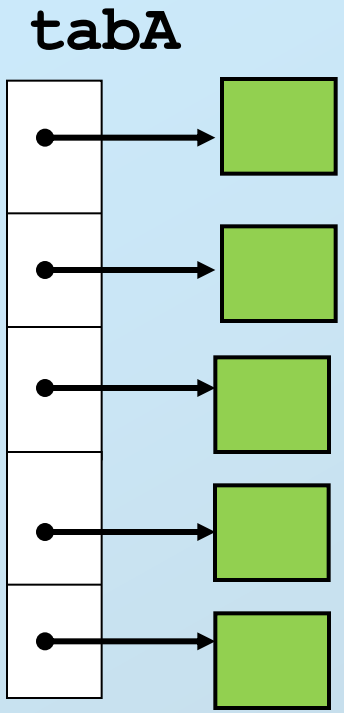
```
int[][] tabA = new int[1][5];
int[] tabB = new int[5];
```

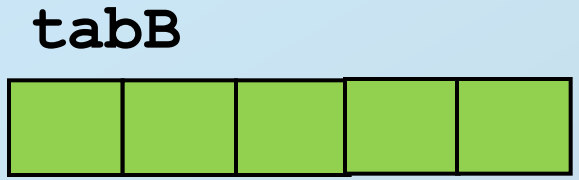
- **tabA** και **tabB** είναι διαφορετικών τύπων:
  - Ο **tabA** είναι δύο διαστάσεων αλλά περιλαμβάνει μόνο μια σειρά με πέντε στοιχεία
  - Ο **tabB** είναι μιας διάστασης και περιλαμβάνει πέντε στοιχεία (είναι μια ατομική σειρά με πέντε στοιχεία)
- Όμως ο (μονοδιάστατος) πίνακας **tabA[0]** έχει τον ίδιο τύπο με τον **tabB**

Σημ: *Βεβαίως θα πρέπει να υπάρχει ειδικός λόγος για να ορισθεί πίνακας δύο διαστάσεων, όπου η εξωτερική διάσταση είναι μόνο ένα (1) στοιχείο*

Συνέχεια επισημάνσεων σε σχέση με τύπους

```
int[][] tabA = new int[5][1];
int[] tabB = new int[5];
```

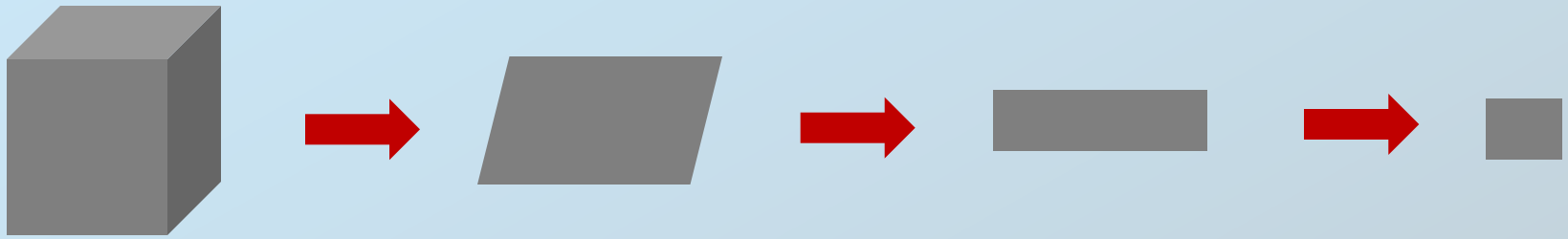




- Ο **tabA** είναι δύο διαστάσεων
  - Έχει πέντε σειρές, όπου η κάθε σειρά έχει ένα μόνο στοιχείο
  - Ουσιαστικά είναι μια στήλη
  - *Απευθείας πρόσβαση έχουμε μόνο στις σειρές*
- Ο **tabB** είναι όπως προηγουμένως, δηλαδή μιας διάστασης πίνακας με πέντε στοιχεία (ουσιαστικά μια σειρά)
- Οι πίνακες **tabA[0]** (σειρά από ένα στοιχείο) και **tabB** (σειρά από πέντε στοιχεία) δεν είναι του ίδιου τύπου

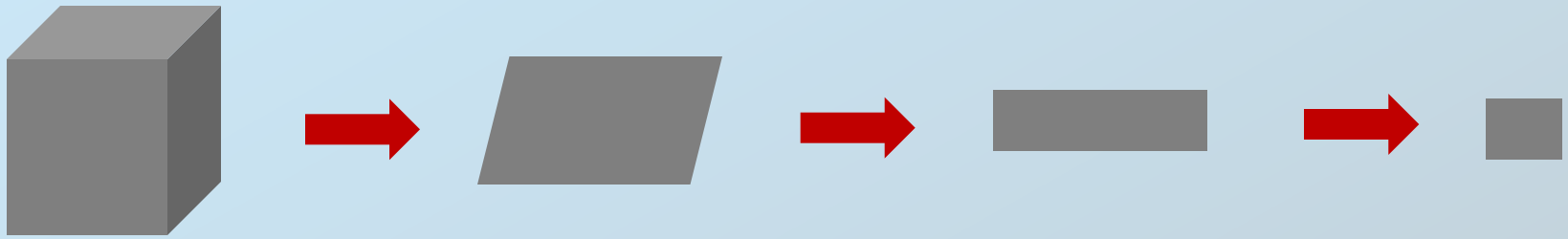
Παράδειγμα: Μέθοδος sum

```
public static double sum (double[][][] xsss){
 double s = 0.0;
 for (int i = 0; i < xsss.length; i++) // for each plane
   for (int j = 0; j < xsss[0].length; j++) // for each row
     for (int k = 0; k < xsss[0][0].length; k++) // for each col
         s += xsss[i][j][k];
 return s;
}
```



Παράδειγμα: Μέθοδος sum

```
public static double sum (double[][][] xsss){
 double s = 0.0;
 for (double[][] xss : xsss) // for each plane
   for (double[] xs : xss) // for each row
     for (double x : xs) // for each col
         s += x;
 return s;
}
```

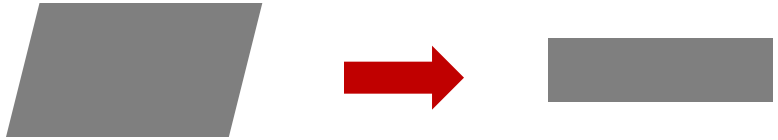


Τρεις εκδοχές της sum

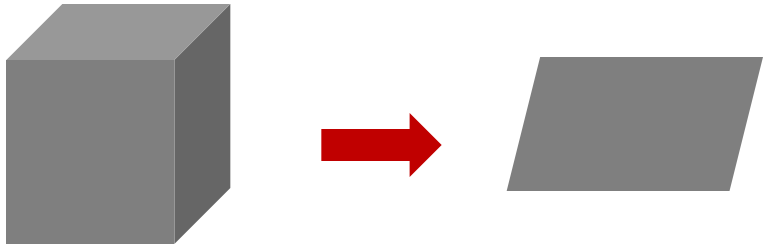
```
public static double sum (double[] xs){
 double s = 0.0;
 for (double x : xs)
     s += x;
 return s;
}
```



```
public static double sum (double[][] xss){
 double s = 0.0;
 for (double[] xs : xss)
     s += sum(xs);
 return s;
}
```



```
public static double sum (double[][][] xsss){
 double s = 0.0;
 for (double[][] xss : xsss)
     s += sum(xss);
 return s;
}
```

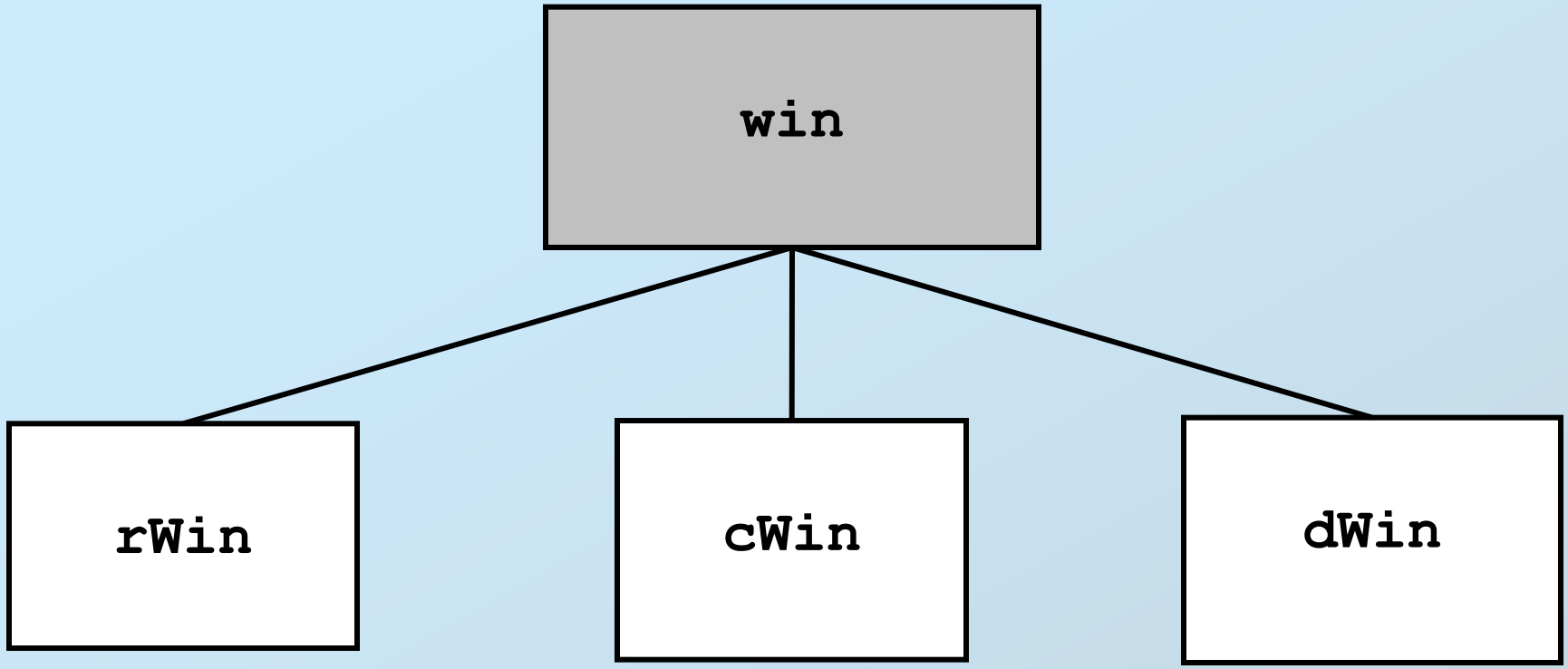


Πρόβλημα: Να ορισθεί η συνάρτηση **`win`** η οποία λαμβάνει ως παράμετρο δεδομένο πίνακα που αντιπροσωπεύει τη διάταξη των επιλογών των δύο παιχτών στο παιγνίδι tic-tac-toe, και επιστρέφει true εάν η διάταξη αποτελεί νίκη για κάποιον από τους παίκτες. Διαφορετικά επιστρέφει false. Σε περίπτωση νίκης, εκτυπώνει ποιος νίκησε και πώς.

Να εφαρμόσουμε τη στρατηγική της «διάσπασης και σύνθεσης»

Διάσπαση προβλήματος σε υπο-προβλήματα

- Είναι τα περιεχόμενα δεδομένης σειράς τα ίδια;
- Είναι τα περιεχόμενα δεδομένης στήλης τα ίδια;
- Είναι όλα τα στοιχεία της μίας ή της άλλης διαγώνιου τα ίδια;




```
public static boolean win (char [][] ttt_b){
 return rWin(ttt_b) || cWin(ttt_b) || dWin(ttt_b);
}
```

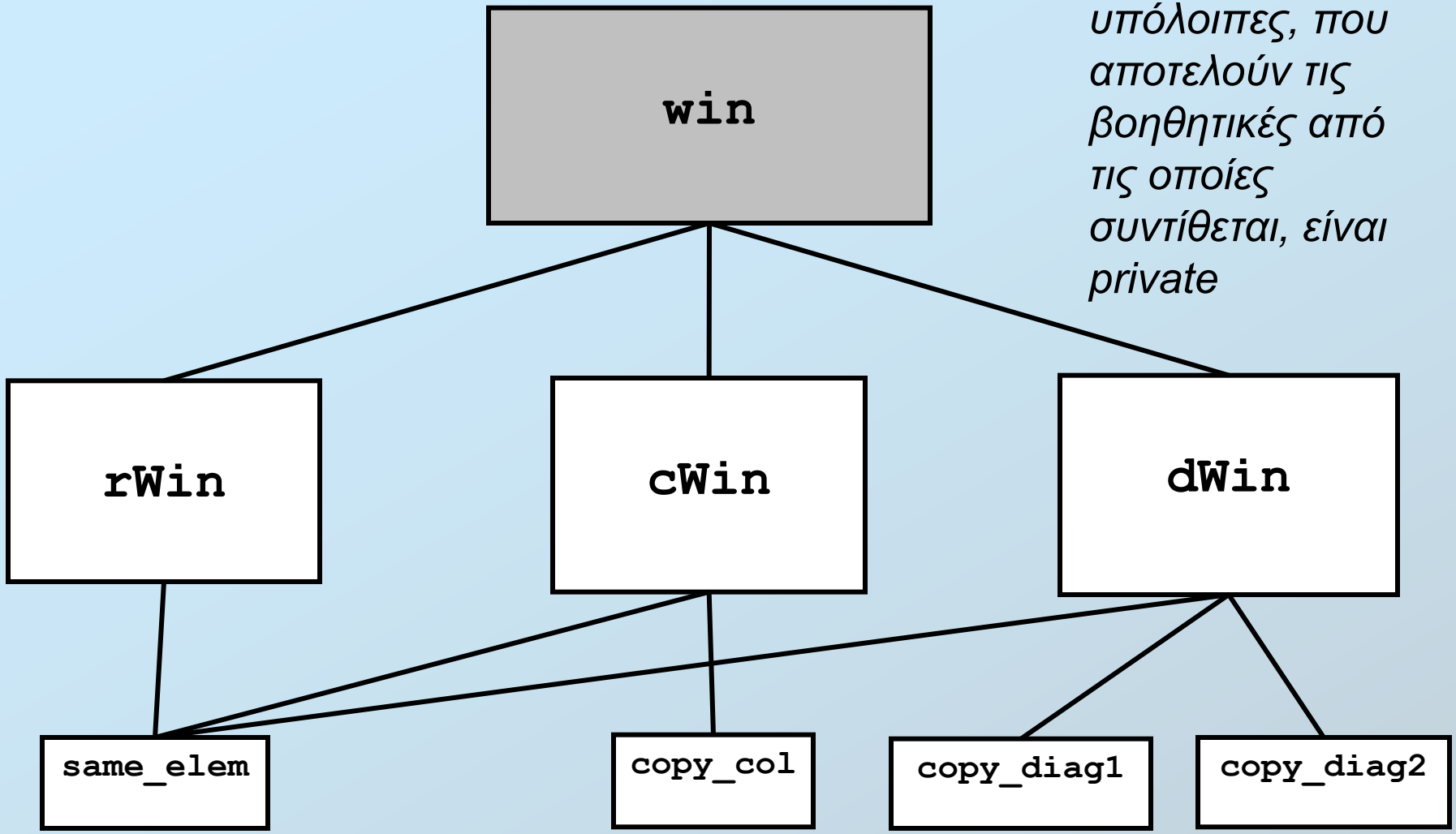
Σχεδιασμός/διάσπαση από τα πάνω προς τα κάτω

Προσδιορισμός πιο βασικών υπο-προβλημάτων

- Είναι όλα τα στοιχεία δεδομένου μονοδιάστατου πίνακα τα ίδια;
- Αντιγραφή δεδομένης στήλης δυσδιάστατου πίνακα σε μονοδιάστατο πίνακα
- Αντιγραφή δεδομένης διαγωνίου δυσδιάστατου πίνακα σε μονοδιάστατο πίνακα

Σημείωση: Μόνο τα στοιχεία της εξωτερικής διάστασης μπορεί να αναφερθούν ως ολότητες, σε σχέση με πολυδιάστατους πίνακες. Επομένως σχετικά με δισδιάστατους πίνακες, μπορεί να γίνει άμεση αναφορά στις σειρές του πίνακα, ως ολότητες (δηλαδή ως μονοδιάστατους πίνακες), αλλά όχι στις στήλες ή τις διαγώνιους του.

Μόνο η win είναι public. Οι υπόλοιπες, που αποτελούν τις βοηθητικές από τις οποίες συντίθεται, είναι private



Υλοποίηση/σύνθεση από κάτω προς πάνω

```
public class TicTacToe {

 // Are all elements of table the same?
 private static boolean same_elem (char[] table) {
   for (int i = 1; i < table.length; i++)
       if (table[i] != table[0]) return false;
   return true; }

 // Copy column col of table into array col_table
 private static void copy_col(char[][] table, char[] col_table,
                              int col){
    for (int i = 0; i < table[0].length; i++)
        col_table[i] = table[i][col];
 }

 // Copy diagonal of table
 private static void copy_diag1(char[][] table, char[] diag){
     for (int i =0; i < diag.length; i++)
         diag[i] = table[i][i];
 }

 // Copy anti-diagonal of table
 private static void copy_diag2(char[][] table, char[] diag){
    for (int i =0; i < diag.length; i++)
        diag[i] = table[i][2-i];
 }
// class definition continues …
```

```
// continuation of class definition

 // Is there a row win?
 private static boolean rWin (char[][] ttt){
   for (int i = 0; i < ttt.length; i++)
       if (ttt[i][0] != 'E' && same_elem(ttt[i])){
           System.out.println("Player " + ttt[i][0] + " wins on row " + (i+1));
           return true;
       }
    return false;
 }

 // Is there a colum win?
 private static boolean cWin (char[][] ttt){
   for (int i = 0; i < ttt.length; i++){
        char[] aux = new char[3];
        copy_col(ttt, aux, i);
        if (aux[0] != 'E' && same_elem(aux)){
           System.out.println("Player " + aux[0] + " wins on colum " + (i+1));
           return true;
          }
    }
    return false;
 }
// class definition continues . . .
```

```
// continuation of class definition

  // Is there a diagonal win?
  private static boolean dWin(char[][] ttt){
    char[]aux = new char[3];
    copy_diag1(ttt, aux);
    if (aux[0] != 'E' && same_elem(aux)){
        System.out.println("Player " + aux[0] + " wins on diagonal");
         return true;
   }
   copy_diag2(ttt, aux);
   if (aux[0] != 'E' && same_elem(aux)){
      System.out.println("Player " + aux[0] + " wins on antidiagonal");
         return true;
   }
   return false;
 }

 // The principal function win
 public static boolean win (char [][] ttt_b){
   return rWin(ttt_b) || cWin(ttt_b) || dWin(ttt_b);
 }

// class definition continues . . .
```

```
// completion of class definition by including main for
// testing purposes

public static void main(String[] args){
   char[][] board = new char[3][3];
   for (int i = 0; i < 3; i++)
      for (int j = 0; j < 3; j++){
          board[i][j]= args[0].charAt(i*3+j);}

   if (win(board)) System.out.println("Thus there is a winner!!");
   else System.out.println("Nobody wins!");
 }

}
```

```
% java TicTacToe EEXEEOEEE
Nobody wins!

% java TicTacToe XEEXEEXEE
Player X wins on col 1
Thus there is a winner!

% java TicTacToe EEOEOEOEE
Player O wins on antiDiagonal
Thus there is a winner!
```

Πρόβλημα: Περπατώ, αποφεύγοντας προηγούμενα σημεία (Self-avoiding walk)

Μοντέλο

Ένα πλέγμα N × N.

Αρχίζω από το μεσαίο σημείο.

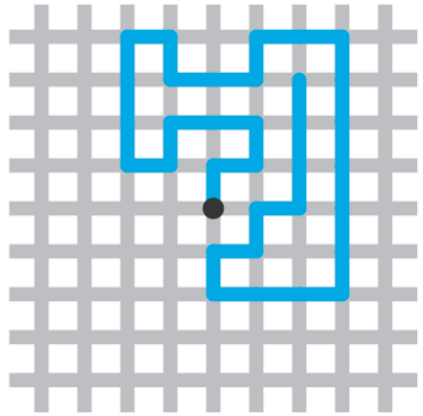
Κινούμαι τυχαία σε κάποια γειτονική διασταύρωση, αποφεύγοντας διασταυρώσεις από τις οποίες έχω ήδη περάσει (ένα βήμα πάνω, κάτω, δεξιά ή αριστερά).

Κατορθώνω να δραπετεύσω ή εγκλωβίζομαι; Δραπετεύω όταν φτάσω σε ακραίο σημείο του πλέγματος (πρώτη/τελευταία σειρά, ή πρώτη/τελευταία στήλη). Εγκλωβίζομαι όταν δεν μπορώ να προχωρήσω από μη-ακραίο σημείο.

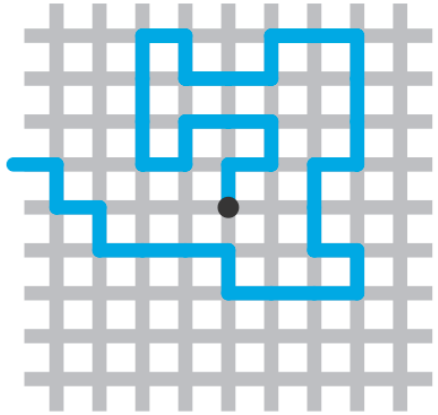
Εφαρμογές

Πολυμερή, Στατιστική Μηχανική, κτλ.

dead end



escape



Ερωτήματα προς διερεύνηση

Ποιο είναι το ποσοστό επιτυχίας μου σε ένα πλέγμα 5 × 5;

Σε ένα πλέγμα N × N;

Σε ένα πλέγμα N × N × N;

Από την αλγοριθμική σκοπιά ποια ερωτήματα πρέπει να απαντηθούν (με άλλα λόγια ποια είναι τα βασικά υπο-προβλήματα που χρειάζεται να επιλυθούν);

Για δεδομένη προσπάθεια

Πώς καταγράφω την πορεία μου, για να γνωρίζω τα σημεία που έχω ήδη επισκεφτεί;

Πώς αντιλαμβάνομαι ότι έχω κατορθώσει να δραπετεύσω, ή το αντίθετο ότι έχω εγκλωβιστεί, για να τερματίσω την προσπάθεια;

Σε περίπτωση που δεν έχω ακόμη εγκλωβιστεί/δραπετεύσει, πώς επιλέγω με τυχαίο τρόπο την επόμενη μου κίνηση;

Αλγοριθμική δομή: απροσδιόριστη επανάληψη (συνεχίζω κανονικά την προσπάθεια μου, δηλαδή την πορεία μου, ενόσω δεν έχω ακόμη κατορθώσει να δραπετεύσω. Διακόπτω όμως ανώμαλα την πορεία μου, μόλις εγκλωβιστώ, εάν και εφόσον αυτό συμβεί – με άλλα λόγια επιλογή εντός της επανάληψης ως προς το ενδεχόμενο πρόωρου τερματισμού της επανάληψης)

Για δεδομένο σύνολο προσπαθειών

Διαδοχικά εκπληρώνω το δεδομένο σύνολο προσπαθειών που έχω στη διάθεσή μου, αρχίζοντας κάθε προσπάθειά μου από το μεσαίο σημείο του πλέγματος, και παράλληλα καταγράφω τις επιτυχίες (δραπέτευσα!) και αποτυχίες μου (εγκλωβίστηκα!).

Στο τέλος παρουσιάζω το ποσοστό αποτυχιών (ή επιτυχιών) μου.

Αλγοριθμική δομή: απαριθμητή επανάληψη

Αλγόριθμος λύσης σε υψηλό επίπεδο ανάλυσης δίνοντας όμως την ουσία

Αρχικοποιώ τον μετρητή μου για τις αποτυχημένες προσπάθειες στο 0

Επαναλαμβάνω από την πρώτη μέχρι την τελευταία προσπάθεια που έχω

Αρχικοποιώ το πλέγμα μου για να φαίνεται ότι δεν έχω ακόμη ξεκινήσει την τρέχουσα προσπάθειά μου και τοποθετώ τον εαυτό μου στο μεσαίο σημείο

Επαναλαμβάνω ενόσω δεν έχω ακόμη κατορθώσει να δραπετεύσω

Καταγράφω στο πλέγμα το σημείο που βρίσκομαι.

Αν δυστυχώς έχω εγκλωβιστεί, καταγράφω στο μετρητή μου την αποτυχημένη προσπάθεια και διακόπτω (ανώμαλα)τη συγκεκριμένη προσπάθεια!

πρόωρη διακοπή και έξοδος από τον εσωτερικό βρόγχο

Επιλέγω τυχαία, ανάμεσα στις πιθανές επιλογές που έχω, το επόμενο σημείο στο οποίο θα πάω

Παρουσιάζω το ποσοστό των αποτυχιών μου!

Διατύπωση βημάτων αλγορίθμου σε (ψευδο)κώδικα

```
int deadEnds = 0;  // no of unsuccessful tries
for (t = 0; t < T; t++) { // I repeat for each try
    boolean[][] a = new boolean[N][N]; // grid representation
    int x = N/2, y = N/2;
    while (x > 0 && x < N-1 && y > 0 && y < N-1) { // I have not escaped yet
            a[x][y] = true; // I am at this point
            if (a[x+1][y] && a[x-1][y] && a[x][y+1] && a[x][y-1]){
                // Alas . . . I am trapped!
                deadEnds++; break;} // I must interrupt this try!
            // I can proceed . . .
            double r = Math.random(); // . . but in a random way
            if      (r < 0.25) {if (!a[x+1][y]) x++;}
            else if (r < 0.50) {if (!a[x-1][y]) x--;}
            else if (r < 0.75) {if (!a[x][y+1]) y++;}
            else if (r < 1.0)  {if (!a[x][y-1]) y--;}

    }
}
System.out.println(100*deadEnds/T + "% of dead ends");
```

Πλήρης Υλοποίηση Αλγορίθμου

```
public class SelfAvoidingWalk {
   public static void main(String[] args) {
      int N = Integer.parseInt(args[0]);     // grid size
      int T = Integer.parseInt(args[1]);     // number of trials
      int deadEnds = 0;                      // trials resulting in dead end

      for (int t = 0; t < T; t++) {
         boolean[][] a = new boolean[N][N];  // intersections visited
         int x = N/2, y = N/2;               // current position

         while (x > 0 && x < N-1 && y > 0 && y < N-1)  {
            if (a[x-1][y] && a[x+1][y] && a[x][y-1] && a[x][y+1]) {
               deadEnds++;
               break;
            }
            a[x][y] = true;                  // mark as visited
            double r = Math.random();
            if      (r < 0.25) { if (!a[x+1][y]) x++; }
            else if (r < 0.50) { if (!a[x-1][y]) x--; }
            else if (r < 0.75) { if (!a[x][y+1]) y++; }
            else if (r < 1.00) { if (!a[x][y-1]) y--; }
         }
      }
      System.out.println(100*deadEnds/T + "% dead ends");
   }
}
```

Σχολιάστε την ακόλουθη εντολή επιλογής

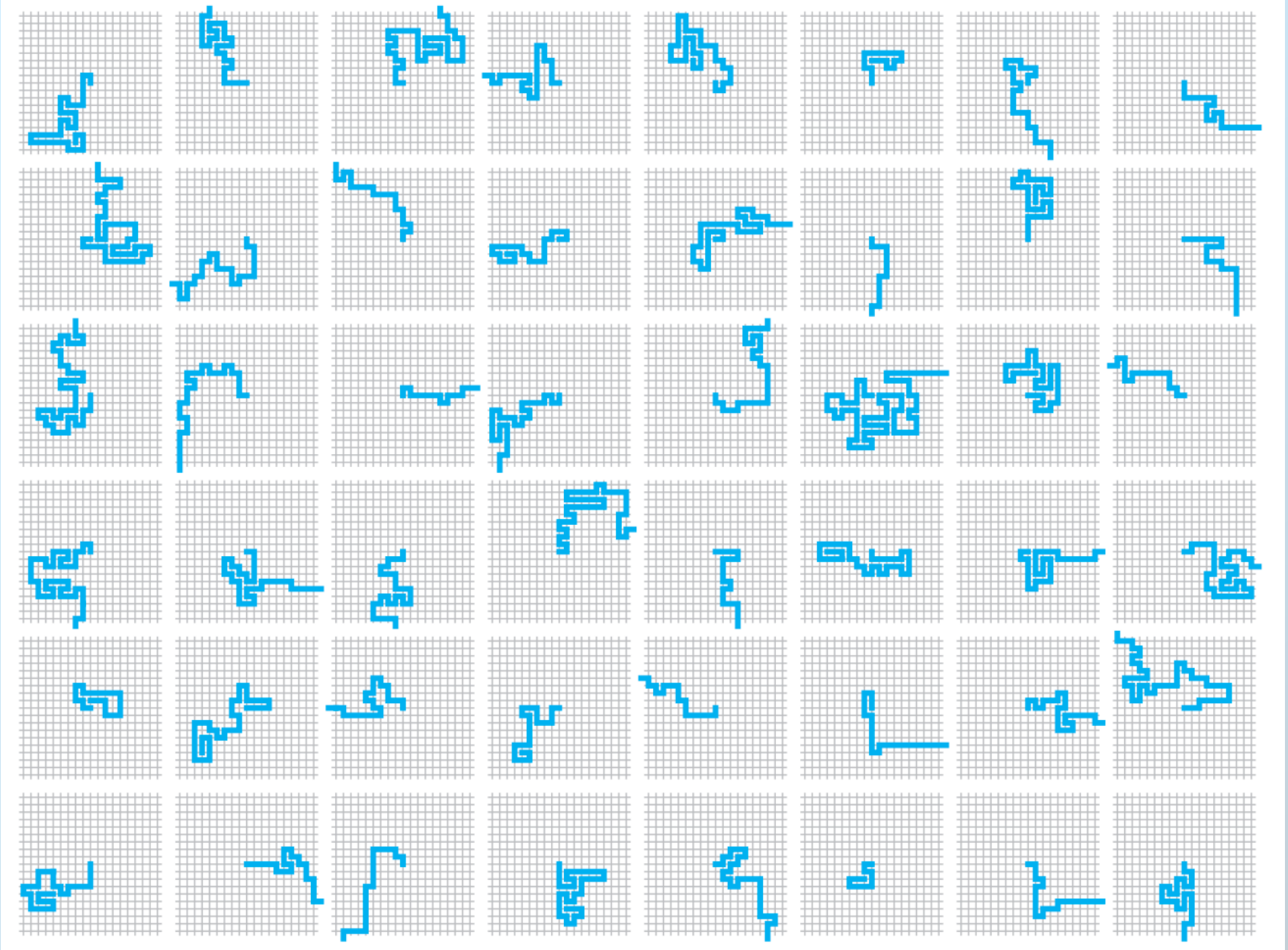
(στο πλαίσιο της απροσδιόριστης επανάληψης)

	0	1	2	3	4	5
0						
1						
2						
3			true	true		
4						
5						

```
double r = Math.random();
if      (r < 0.25) {if (!a[x+1][y]) x++;}
else if (r < 0.50) {if (!a[x-1][y]) x--;}
else if (r < 0.75) {if (!a[x][y+1]) y++;}
else if (r < 1.0)  {if (!a[x][y-1]) y--;}
```

Έστω ότι, σε ένα δεδομένο σημείο της εκτέλεσης, οι είσοδοι **`a[x+1][y]`**, **`a[x][y+1]`** και **`a[x][y-1]`** είναι **`true`** και η μεταβλητή **`r`** διαδοχικά λαμβάνει τις τιμές 0.1, 0.045, 0.83, 0.95, 0.8, και 0.62

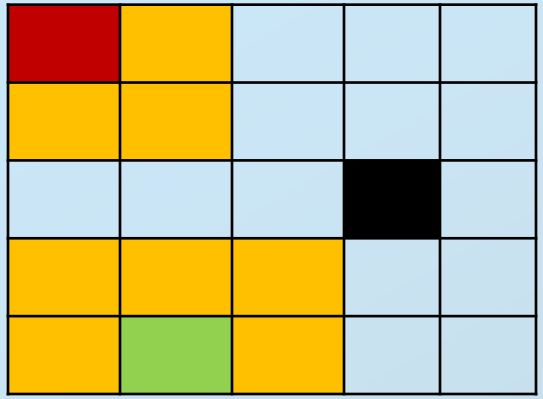
Παραδείγματα Self-Avoiding Walks



Πρόβλημα: Το παιχνίδι της ζωής του Conway

- Ένα κελί αναπαρίσταται από ένα μπλοκ σε ένα πλέγμα
- Κάθε κελί έχει (μέχρι) 8 γείτονες
- Απλοί κανόνες για ένα κελί όταν “έρχεται στη ζωή” ή όταν “πεθαίνει”:
  - Ένα ζωντανό κελί με λιγότερους από 2 ζωντανούς γείτονες πεθαίνει από μοναξιά.
  - Ένα ζωντανό κελί με περισσότερους από 3 ζωντανούς γείτονες πεθαίνει από υπερπληθυσμό.
  - Ένα ζωντανό κελί με ακριβώς 2 ή 3 ζωντανούς γείτονες παραμένει ζωντανό.
  - Ένα νεκρό κελί με ακριβώς 3 ζωντανούς γείτονες έρχεται στη ζωή.

Υπό-πρόβλημα: Πόσους ζωντανούς γείτονες έχει δεδομένο κελί;



Αναπαράσταση πλέγματος N x N

```
boolean[][] Gen = new boolean[N][N];
```

```
// How many live neighbors does cell [x][y] have?

public static int live_neighbors(boolean[][] Gen,
                                 int x, int y){
  int N = Gen.length;
  int count = 0;
  if (Gen[x][y]) count--;
  for (int i = x-1; i <= x+1; i++)
   for (int j = y-1; j <= y+1; j++)
      if (i >= 0 && i < N && j >= 0 && j < N &&
          Gen[i][j]) count++;
  return count;
 }
```

Υπό-πρόβλημα: Αρχικοποίηση πλέγματος (1η Γενεά κελιών)

```
// Initialize first generation randomly -
// p is the probability that a cell is alive

public static void initGen(boolean[][] Gen, double p){
   int N = Gen.length;
   for (int i = 0; i < N; i++)
    for (int j = 0; j < N; j++)
      if (Math.random() < p) Gen[i][j] = true;
  }
```

Υπό-πρόβλημα: Δημιουργία νέας Γενεάς από την τρέχουσα Γενεά

```
// Compute and return new generation from current
// generation

public static boolean[][] nextGen(boolean[][] Gen){
   int N = Gen.length;
   boolean[][] NGen = new boolean[N][N];
   for (int i = 0; i < N; i++)
    for (int j = 0; j < N; j++){
     int n = live_neighbors(Gen, i, j);
     if (Gen[i][j] && (n < 2 || n > 3))
        NGen[i][j]=false;
     else if (!Gen[i][j] && n==3) NGen[i][j]=true;
     else NGen[i][j] = Gen[i][j];
    }
  return NGen;
 }
```

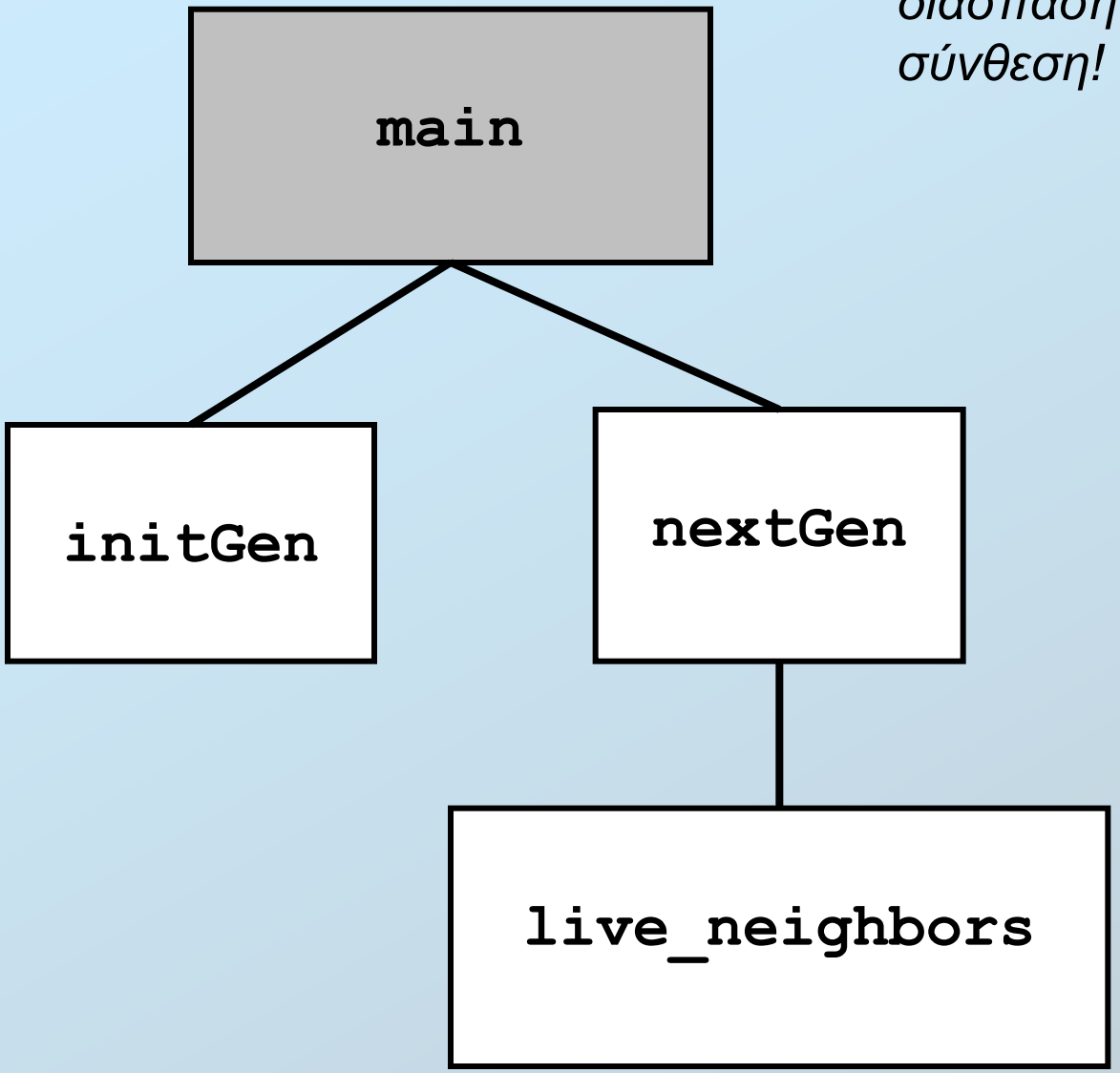
Συνθέτοντας την main

```
public static void main(String[] args){
  // N - grid size
  int N = Integer.parseInt(args[0]);
  // p - probability of a cell being initially alive
  double p = Double.parseDouble(args[1]);
  // T - number of next generations to be computed
  int T = Integer.parseInt(args[2]);

  boolean[][]Generation = new boolean[N][N];
  initGen(Generation, p);
  for (int t = 1; t <= T; t++){
       Generation = nextGen(Generation);
  }
 }
```

Τι απουσιάζει από το πρόγραμμα;

Και πάλι διάσπαση και σύνθεση!

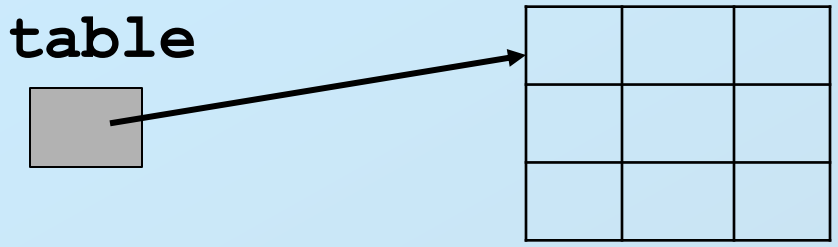


Σύνοψη

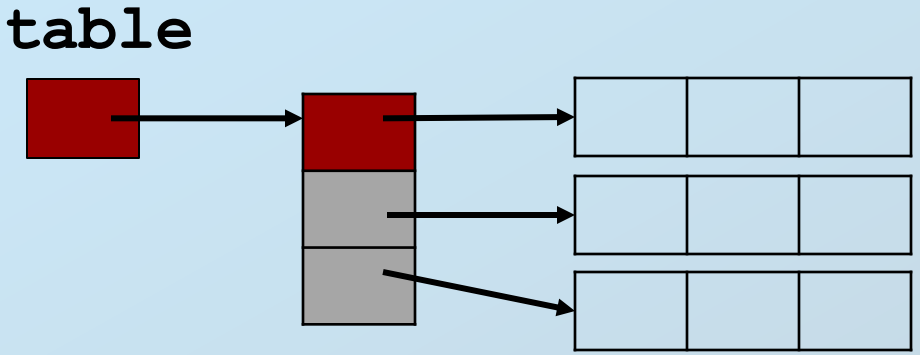
- Παραδείγματα χρήσης δισδιάστατων πινάκων
  - Απευθείας πρόσβαση σε σειρές, αλλά όχι σε στήλες
- Οι πίνακες αποτελούν
  - Ένα οργανωμένο τρόπο αποθήκευσης τεράστιων ποσοτήτων δεδομένων
  - Σχεδόν το ίδιο εύκολοι στη χρήση τους, όπως και οι βασικοί τύποι δεδομένων
- Ανάλυση πινάκων πολλαπλών διαστάσεων
  - Σε σχέση με την κάθε διάσταση και όχι μόνο σε σχέση με τα στοιχεία που αφορούν την τελευταία διάσταση
  - Συνεπώς διάσπαση σε πολλαπλά επίπεδα σύμφωνα με τις διαστάσεις ενός πίνακα

Πίνακες δύο διαστάσεων

```
int[][] table = new int[3][3];
```



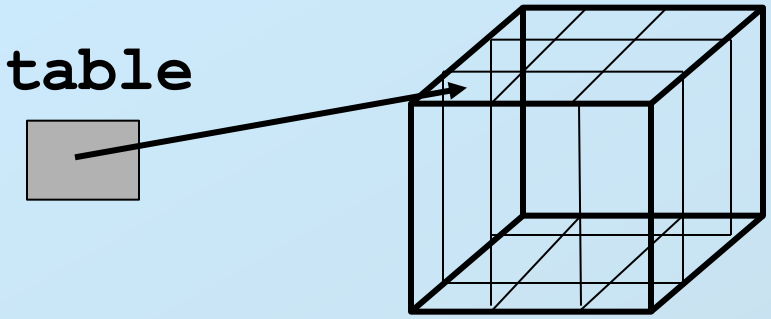
Τύπος σειρών table: `int[]`
Τύπος ατομικών στοιχείων table: `int`



Πλήθος σειρών table: `table.length`
Πλήθος στηλών table: `table[0].length`

Πίνακες τριών διαστάσεων

```
int[][][] table = new int[3][3][3];
```



Τύπος στοιχείων πρώτης διάστασης table: `int[][]`

- *Πίνακες δύο διαστάσεων (συστάδες)*
- ***`table.length`*** *δίνει το πλήθος των εν λόγω συστάδων*
- ***`table[0].length`*** *δίνει το πλήθος των σειρών της κάθε συστάδας*
- ***`table[0][0].length`*** *δίνει το πλήθος των στηλών της κάθε συστάδας*

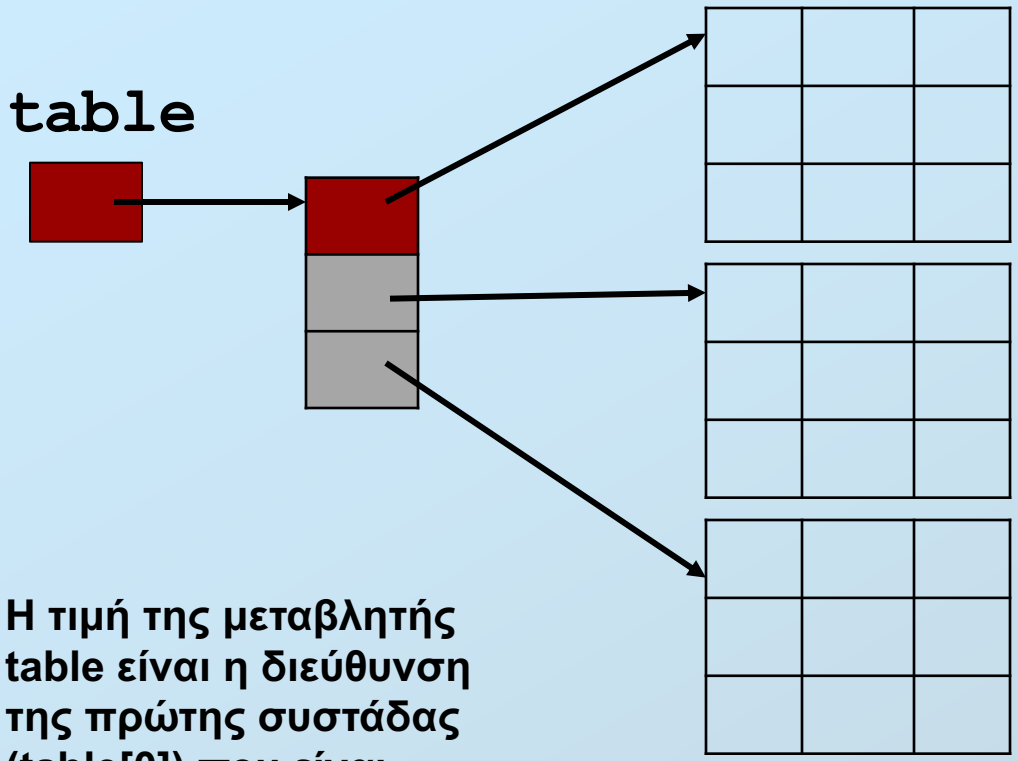
Τύπος στοιχείων δεύτερης διάστασης table: `int[]`

- *Πίνακες μίας διάστασης*

Τύπος στοιχείων τρίτης διάστασης table: `int`

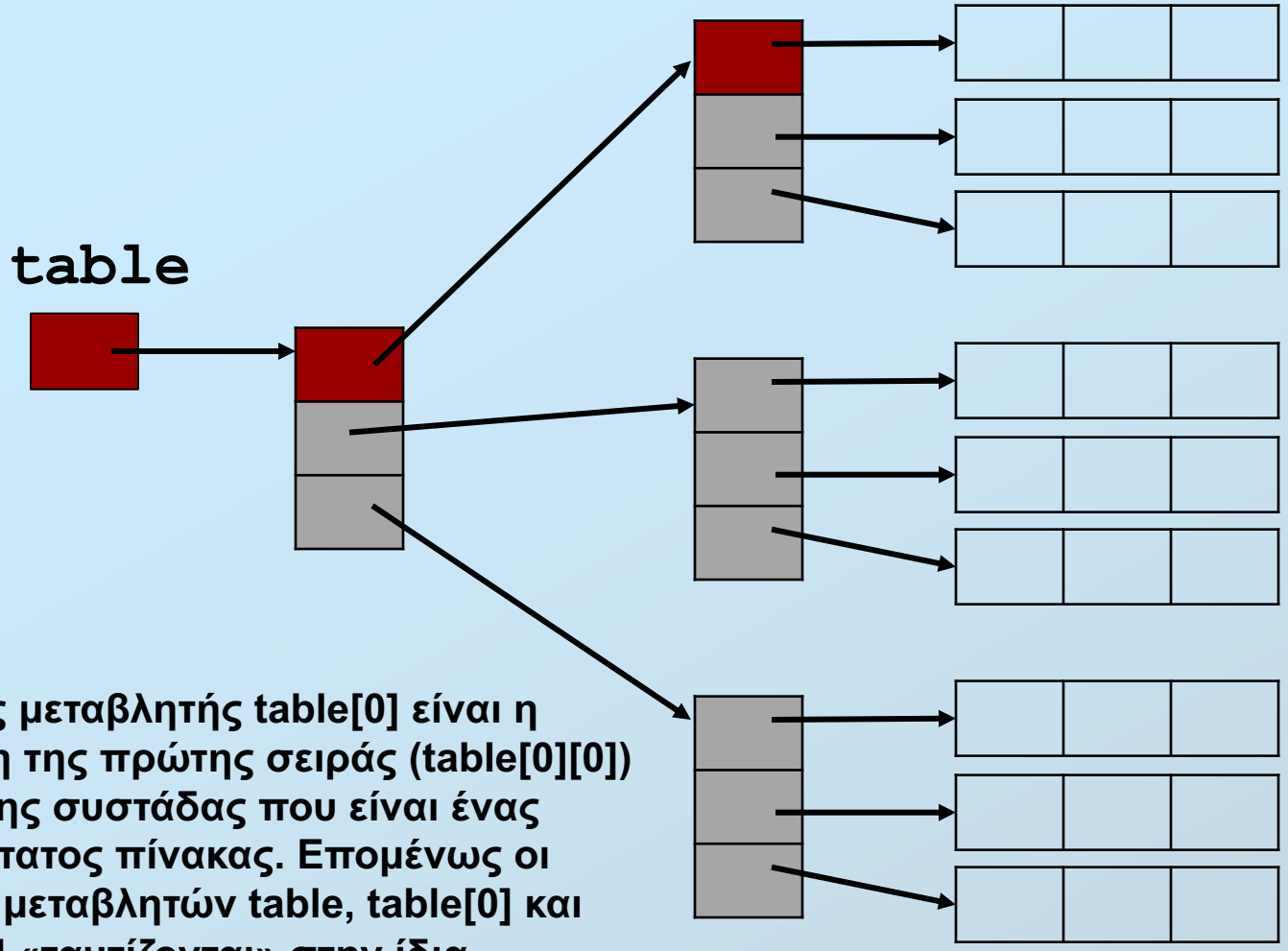
- *Ατομικά στοιχεία*

Ανάλυση/διάσπαση βάσει της πρώτης διάστασης του πίνακα



Η τιμή της μεταβλητής table είναι η διεύθυνση της πρώτης συστάδας (table[0]) που είναι ένας δισδιάστατος πίνακας

Ανάλυση/διάσπαση βάσει της πρώτης και δεύτερης διάστασης του πίνακα



Η τιμή της μεταβλητής table[0] είναι η διεύθυνση της πρώτης σειράς (table[0][0]) της πρώτης συστάδας που είναι ένας μονοδιάστατος πίνακας. Επομένως οι τιμές των μεταβλητών table, table[0] και table[0][0] «ταυτίζονται» στην ίδια διεύθυνση, που είναι η διεύθυνση του πρώτου σε σειρά ατομικού στοιχείου του table. Η μεταβλητή table[0][0][0], όπως και πιο γενικά οι μεταβλητές table[i][j][k] είναι απλές αριθμητικές τιμές.

Στο βιβλίο

- **Ενότητα 1.4**